AutoRL for Predictive Maintenance on the Edge: A Comparative Study of REINFORCE with Attention Mechanisms for Milling Tool Replacement

Rajesh Siraskar
PhD Researcher, Predictive Maintenance and Reinforcement Learning
(Affiliation as appropriate)
(email as appropriate)

March 2026

Abstract

Industry 4.0 maintenance systems increasingly rely on edge-computing deployments where stringent limits on memory, latency, and energy consumption favor computationally light learning methods. In this work we study *AutoRL*, an automated reinforcement learning (RL) pipeline that trains and evaluates maintenance agents for a milling machine tool-replacement task, with particular focus on whether the naïve Monte-Carlo policy-gradient algorithm REINFORCE can be competitive with more advanced methods such as PPO. The contribution is threefold: (i) an end-to-end AutoRL training and multi-round evaluation framework, (ii) a Gymnasium-compatible predictive-maintenance environment that hides tool wear from observations to enforce inference from sensor signals alone, and (iii) a systematic comparison across four RL algorithms (A2C, DQN, PPO, REINFORCE) and four attention mechanisms (Nadaraya–Watson, Temporal, Multi-Head, Self-Attention) integrated as Stable-Baselines3 feature extractors.

Across 48 trained agents (3 training files $\times$ 4 algorithms $\times$ 4 attention mechanisms) and 20 evaluation rounds per configuration, tested on six unseen datasets, REINFORCE with temporal or multi-head attention achieves the highest mean evaluation score and is statistically superior to non-REINFORCE baselines (one-tailed Welch t-test, $p \ll 0.05$). The results suggest that, when paired with inductive biases from attention feature extractors, lightweight policy-gradient methods can match (and sometimes exceed) heavier optimization schemes while aligning with sustainability and carbon-footprint objectives.

Keywords: Predictive maintenance; Industry 4.0; Edge computing; Reinforcement learning; REINFORCE; PPO; Attention mechanisms; Milling machine; Sustainable AI.

1 Context: Industry 4.0/5.0, predictive maintenance, and edge constraints

Predictive maintenance (PdM) is a foundational Industry 4.0 capability: instrumented assets continuously produce sensor telemetry and maintenance decisions are optimized to reduce unplanned downtime and extend component life. In practical deployments, PdM must increasingly operate at the *edge*—near machines and shop-floor controllers—to meet latency, bandwidth, and reliability constraints, and to reduce operational risk associated with centralized compute.

Edge deployment imposes hard resource limits (compute cycles, memory footprint, and energy), motivating *lightweight* ML/RL models and training procedures. These constraints directly relate to

sustainability considerations: lower compute and shorter training runs reduce electricity consumption and associated carbon emissions, which is increasingly relevant in corporate carbon accounting and Industry 5.0 human-centric and sustainable manufacturing initiatives. Recent work in "Green AI" emphasizes reporting and minimizing computational cost as a first-class metric alongside accuracy [9, 10].

Beyond operational cost, reducing computational demand supports organizational carbon-footprint targets and can strengthen evidence bases used in carbon-credit and ESG reporting workflows, making "efficient learning" practically consequential rather than purely academic.

In this setting, a key research question emerges:

> *Can simple, computationally light RL algorithms such as* REINFORCE *be competitive with more advanced policy-optimization methods (e.g.,* PPO*) for PdM decision-making, especially when augmented with attention-based feature extraction?*

This study builds directly on prior findings that naïve REINFORCE can perform strongly for PdM in untuned regimes and that automated RL frameworks can help industrial practitioners explore algorithm–hyperparameter design spaces [11]. It also leverages prior work applying a Nadaraya–Watson estimator as a lightweight attention mechanism for PdM, motivated by edge constraints [12].

2 AutoRL overview

2.1 Motivation and design goals

AutoRL is designed as a practical experimentation platform to (i) train multiple RL agents over a grid of algorithmic choices and hyperparameters, (ii) evaluate agents on multiple test files using prognostic-aware metrics, and (iii) generate standard plots (heatmaps and analysis summaries) for model selection.

The implementation in this repository provides both a Streamlit application and a CLI pipeline. The CLI (`train_agent.py`) supports training, discovery of trained models, and evaluation in single-round or multi-round modes. Multi-round evaluation produces a "long" CSV with all repetitions to enable statistical analysis (e.g., t-tests).

2.2 Experimental automation and reproducibility

The pipeline supports:

- training across algorithms and attention types (feature extractor plug-ins),
- evaluation across multiple datasets with repeated rounds (here: 20 rounds),
- checkpointing and retry logic using a lightweight SQLite queue to recover from failures during long experiments.

In addition to model files, each trained agent writes a metadata JSON containing algorithm, attention mechanism, training file, learning rate, discount factor, number of episodes, and training summaries.

3 Predictive maintenance as a Markov decision process

3.1 MDP and POMDP formulation

We model the maintenance decision problem as an episodic Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$ [1], where $\mathcal{S}$ is the set of environment states, $\mathcal{A}$ is the action space, P is the transition kernel, r is the reward function, and γ is the discount factor. At each discrete timestep t, the agent chooses an action

$$a_t \in \mathcal{A} = \{0, 1\}, \tag{1}$$

where $a_t = 0$ denotes *replace* and $a_t = 1$ denotes *continue*. The environment evolves along a recorded run-to-failure dataset where the latent degradation variable (tool wear) is w_t.

Hidden states and partial observability. In classical MDPs, the agent observes the full state s_t at each step. However, in real-world predictive maintenance, the true state (e.g., tool wear w_t) is often *hidden* and cannot be measured directly. Instead, the agent receives a partial observation o_t derived from sensor signals. This setting is more accurately described as a *partially observable MDP* (POMDP), defined by $\mathcal{M}' = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \mathcal{O}, O)$, where $\mathcal{O}$ is the observation space and O is the observation function:

$$o_t = O(s_t) = \text{Sensors}(t), \qquad w_t \notin o_t. \tag{2}$$

The agent must infer the hidden state (wear) from the history of observations and actions, motivating the use of feature extractors and attention mechanisms to capture temporal and cross-sensor dependencies.

3.2 Observation design: wear is hidden

A critical modelling choice is that w_t is *excluded* from observations. The agent must infer maintenance decisions solely from sensors (forces, vibrations, acoustics, currents), which aligns with realistic deployments where true wear is expensive or impossible to measure in real time. Formally, the observation is as above:

$$o_t \equiv \mathbf{x}_t = \text{Sensors}(t), \qquad w_t \notin o_t. \tag{3}$$

This makes the environment a simple POMDP, not a fully observable MDP. The state is only partially visible, and the agent must learn to extract relevant information from the available sensor data.

3.3 Episode termination

Episodes terminate on (i) action *replace* or (ii) a threshold violation (wear exceeding the wear threshold), or (iii) end-of-file truncation.

3.4 Reward function

The reward is shaped to encourage (a) long tool usage without violation and (b) replacement close to (but before) the wear threshold. With wear threshold W and wear ratio $\rho_t = w_t/W$, the

environment implements a piecewise reward (see `MT_Env.step` in `rl_pdm.py`):

$$r_t = \begin{cases} R_3 + \kappa(\rho_t) & a_t = 0,\ w_t \le W \\ \frac{1}{2}R_2 & a_t = 0,\ w_t > W \\ R_2 & a_t = 1,\ w_t > W \\ R_1 & a_t = 1,\ w_t \le W \end{cases} \tag{4}$$

where R_1 is a small survival reward, R_2 is a catastrophic violation penalty, R_3 is a small replacement cost, and $\kappa(\rho)$ is a bonus increasing with ρ for high-wear pre-threshold replacement (implemented as a stepped function for $\rho > 0.7, 0.8, 0.9$).

This design encodes an asymmetric risk profile consistent with safety-critical maintenance: a single violation is substantially worse than an "early" replacement.

4 Attention mechanisms for predictive maintenance

4.1 Attention in general

Attention mechanisms allow models to dynamically focus on the most relevant parts of their input. In its general form, attention is a mapping from queries Q, keys K, and values V to a context representation:

$$\mathrm{Att}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{5}$$

where d_k is the key dimension. This formulation, popularized by Transformer architectures [6], enables the model to weigh different input features or time steps according to their relevance.

In predictive maintenance, attention is attractive because it can (i) emphasize salient sensors, (ii) highlight interactions among signals, and (iii) provide a structured inductive bias without necessarily increasing end-to-end model complexity dramatically. In this work, attention is implemented as a *feature extractor* $\phi_\theta(\mathbf{x}_t)$ inside the policy/value networks (Stable-Baselines3 custom extractors), shifting attention from "sequence-to-sequence" NLP settings to "feature weighting/fusion" over multi-sensor inputs.

4.2 Nadaraya–Watson (NW) attention

Theory. The Nadaraya–Watson estimator performs kernel regression by computing a similarity-weighted average over reference points [7, 8]. In an attention view, the query is the current feature vector and keys are prototypes; weights are derived from a kernel over distances.

Implementation in AutoRL. The extractor (`NadarayaWatsonExtractor`) learns: (i) a positive bandwidth parameter $\beta = \exp(\log \beta)$, (ii) a set of learnable keys $\{\mathbf{k}_i\}_{i=1}^m$ and values $\{\mathbf{v}_i\}_{i=1}^m$ (memory/prototypes), and (iii) a linear query projection. For a batch element, the attention weights are

$$\alpha_i = \mathrm{softmax}\big(-\beta \, \|\mathbf{q} - \mathbf{k}_i\|_2^2\big), \tag{6}$$

and the output context is $\sum_i \alpha_i \mathbf{v}_i$.

Pros/cons. NW attention is computationally light (distance + softmax), offers a smooth inductive bias, and connects to prior PdM attention work emphasizing edge constraints [12]. However, prototype-based memories may underfit complex feature interactions when the optimal policy depends on higher-order relationships across sensors.

4.3 Temporal attention (TP)

Theory. Degradation is inherently temporal; positional encodings provide a simple mechanism to inject time context into a feed-forward representation. Sinusoidal encodings are defined as

$$\mathrm{PE}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \tag{7}$$

$$\mathrm{PE}(pos, 2i{+}1) = \cos\left(\frac{pos}{10000^{2i/d}}\right). \tag{8}$$

Implementation in AutoRL. The extractor (`TemporalAttentionExtractor`) uses: (i) a fixed sinusoidal positional encoding buffer, (ii) a learnable temporal embedding vector, (iii) a spatial attention MLP producing a per-feature soft mask, and (iv) a fusion MLP over concatenated weighted spatial features and temporal features. A timestep counter is maintained as a buffer and incremented on each forward pass.

Pros/cons. Temporal attention is lightweight yet adds an explicit monotone-time context, which is often missing in per-timestep sensor vectors. A limitation is that the internal timestep is not automatically reset by the policy network on environment resets; care must be taken in production to align temporal state with episode boundaries.

4.4 Multi-head attention (MH)

Theory. Multi-head attention typically learns multiple subspaces of attention to capture heterogeneous dependencies [6].

Implementation in AutoRL. The extractor (`MultiHeadAttentionExtractor`) projects the observation into queries, keys, and values and reshapes them into H heads. Because observations are single feature vectors (not sequences), the code computes a per-head scalar score

$$s_h = \frac{\langle \mathbf{Q}_h, \mathbf{K}_h \rangle}{\sqrt{d_h}}, \qquad \alpha_h = \mathrm{softmax}(\mathbf{s})_h, \tag{9}$$

then gates each head's value $\mathbf{V}_h$ by α_h and concatenates across heads. This acts as a structured mixture-of-subspaces: different heads can specialize to different sensor groupings or degradation regimes.

Pros/cons. MH adds representational capacity with controlled width and can be efficient on low-dimensional sensor vectors. Since attention is over heads rather than over time indices, interpretability differs from classical temporal self-attention.

4.5 Self-attention (SA)

Theory. Self-attention is a mechanism that allows each feature (or token) to attend to all others, learning dependencies and interactions. In the standard Transformer, self-attention is defined as:

$$\mathrm{SelfAtt}(X) = \mathrm{softmax}\left(\frac{XW_Q(XW_K)^\top}{\sqrt{d_k}}\right) XW_V \tag{10}$$

where $X \in \mathbb{R}^{n\times d}$ is the input (with n features or tokens), and W_Q, W_K, W_V are learned projection matrices. This computes a weighted sum of value vectors for each feature, with weights determined by the similarity of queries and keys.

In our setting, the input X is a batch of projected sensor feature vectors (not a sequence), so self-attention operates across features or across the batch, depending on implementation. This enables the model to learn complex relationships between sensor channels.

Implementation in AutoRL. The extractor (`SelfAttentionExtractor`) projects inputs to a feature space and computes Q, K, V with linear layers. It forms a scalar score via a dot product and applies a softmax normalization across the *batch* dimension, then uses residual connections and a feed-forward network with layer normalization. The batchwise normalization acts as a gating mechanism, but is not equivalent to standard token-wise attention.

Pros/cons. This design can stabilize training via normalization and residual structure, and can model feature interactions. However, batchwise normalization may introduce sensitivity to batch composition and is less interpretable than classical temporal self-attention.

5 RL algorithms studied

We compare four standard RL approaches commonly used for control and decision-making.

5.1 DQN

DQN learns an action-value function $Q_\theta(s, a)$ and minimizes a temporal-difference objective [3]:

$$\mathcal{L}(\theta) = \mathbb{E}\big[\big(r_t + \gamma \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a') - Q_\theta(s_t, a_t)\big)^2\big], \tag{11}$$

where $\bar{\theta}$ denotes a target network.

5.2 A2C

Actor–critic methods learn both a policy $\pi_\theta(a|s)$ and a value baseline $V_\psi(s)$. The advantage estimate

$$\hat{A}_t = \hat{G}_t - V_\psi(s_t) \tag{12}$$

reduces variance in policy gradients. A2C is a synchronous variant of the asynchronous actor–critic family [4].

5.3 PPO

PPO improves stability by constraining policy updates via a clipped surrogate objective [5]:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, \tag{13}$$

$$\mathcal{L}^{\text{clip}}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]. \tag{14}$$

While effective, PPO typically incurs higher compute due to multiple epochs/minibatch passes per batch.

5.4 REINFORCE

REINFORCE [2] estimates the policy gradient from Monte-Carlo returns. For a trajectory $\tau = (s_0, a_0, \ldots, s_T)$ with return

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}, \tag{15}$$

REINFORCE performs gradient ascent on

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} G_t \right], \tag{16}$$

with gradient estimator

$$\nabla_\theta J(\theta) \approx \mathbb{E} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \left(G_t - b(s_t) \right) \right]. \tag{17}$$

Implementation in AutoRL. The code implements an SB3-compatible `REINFORCE` class that uses the SB3 rollout buffer returns and an optional value-function baseline (`use_baseline=True`). The loss corresponds to

$$\mathcal{L}(\theta, \psi) = -\mathbb{E}[\log \pi_\theta(a|s)\, \hat{A}] + \lambda_H\, \mathcal{L}_{\text{ent}} + \lambda_V\, \|\hat{G} - V_\psi(s)\|_2^2, \tag{18}$$

where entropy regularization encourages exploration.

Why REINFORCE can be edge-suited. Compared to PPO, REINFORCE is algorithmically simpler (no ratio clipping, fewer inner optimization epochs) and thus can reduce training-side compute. In an AutoRL setting where many algorithm/attention combinations are trained, such savings compound and can lower overall experimental carbon footprint.

6 Evaluation metrics

6.1 Prognostic-specific λ metric

PdM evaluation requires metrics beyond generic RL return. Following the prognostics-oriented evaluation perspective synthesized in a systematic technical review [13], we use a λ-style timing metric derived from the time difference between a model's first replacement prediction and a threshold-based "end-of-life" point.

In code, the environment defines an "in-advance replacement" (IAR) window around the wear threshold W with range parameter η (`IAR_RANGE`=0.05). The IAR lower bound is

$$w_{\text{IAR}} = W(1 - \eta), \tag{19}$$

and t_λ is the first timestep where $w_t \geq w_{\text{IAR}}$. Let t_{FR} be the first timestep where the agent chooses *replace*. The signed timing metric is

$$\lambda = t_\lambda - t_{\text{FR}}, \tag{20}$$

and the evaluation normalizes $|\lambda|$ so that smaller values are better.

6.2 Composite evaluation score

To compare policies, AutoRL defines a normalized weighted score (`calculate_eval_score`):

$$S = 0.50\, S_U + 0.30\, S_\lambda + 0.20\, S_V, \qquad S \in [0, 1], \tag{21}$$

where S_U is normalized tool usage, $S_\lambda = \max\{0, 1 - |\lambda|/w_{\text{IAR}}\}$, and $S_V = 1/(1 + \text{violations})$ penalizes threshold violations.

7 Experimental setup

7.1 Datasets and generalization protocol

Experiments use the SIT milling-machine sensor schema (8 sensor channels per timestep). Models are trained on three files (`SIT_8`, `SIT_10`, `SIT_12`) and evaluated on six unseen files (`SIT_7`, `SIT_9`, `SIT_11`, `SIT_13`, `SIT_14`, `SIT_15`). This split supports a domain-generalization question: does a learned maintenance policy transfer to unseen runs?

7.2 Model grid

We train $3 \times 4 \times 4 = 48$ agents: algorithms {A2C, DQN, PPO, REINFORCE} crossed with attention types {NW, TP, MH, SA}. Training metadata indicates 500 episodes, learning rate 10^{-4}, and $\gamma = 0.99$ for the reported model set.

7.3 Multi-round evaluation

Each model–test-file pair is evaluated for 20 rounds. The long-form results CSV is retained for statistical analysis; plots and heatmaps use round means. The primary analysis artifacts used in this paper are:

- `results/Analysis/01-Mar-2026 Good Set-1/Evaluation_Results_Multiround_SIT_NoAM_20260301_181`
- `paper/figures/analysis_report_sit.png`,
- `paper/figures/heatmap_allagents_sit.png`.

8 Results

8.1 Overall performance and best configurations

Figure 1 and Figure 2 summarize the evaluation outcomes. The top-performing combinations on unseen test files are REINFORCE+Temporal and REINFORCE+Multi-Head, both achieving mean evaluation score ≈ 0.947 with tight confidence intervals.

8.2 Hypothesis testing: is REINFORCE better?

To quantify whether REINFORCE improves the evaluation score distribution, we perform a one-tailed independent Welch t-test comparing REINFORCE scores against all non-REINFORCE scores (implemented in `Hypotheis_test.ipynb`). The test yields $t = 22.70$ and $p = 1.70 \times 10^{-90}$ at $\alpha = 0.05$, rejecting H_0 in favor of H_1: REINFORCE is significantly better on this evaluation distribution.

Table 1: Top configurations on *unseen* test files (mean ± 95% CI, 20 rounds; $n = 360$ rows per configuration).

Algorithm	**Attention**	**Eval. Score**	**CI**	**Tool use (%)**	λ
REINFORCE	Temporal	0.947	0.002	95.661	−30.000
REINFORCE	Multi-Head	0.946	0.002	95.661	−30.333
PPO	Multi-Head	0.946	0.002	95.661	−30.444
PPO	Temporal	0.732	0.032	81.171	113.444
PPO	NW	0.729	0.032	81.482	109.389

8.3 Why REINFORCE + (MH/TP) can match or exceed PPO

Empirically, the top three configurations are within a narrow band (Table 1). We propose the following mechanisms explaining why REINFORCE+MH/TP can be comparable to PPO in this setting:

- **Inductive bias dominates optimizer sophistication.** When wear is hidden, the primary challenge is representation learning from sensors. MH and TP extractors inject structure (subspace gating and time-context fusion), reducing the burden on the policy optimizer.
- **Small action space and short-horizon decision.** With $|\mathcal{A}| = 2$ and termination on replacement, the control problem can be closer to learning a boundary in feature space than to long-horizon continuous control, reducing the relative advantage of PPO-style constrained updates.
- **Variance reduction via baseline.** Although termed "naïve", the implementation uses a value-function baseline (by default) which reduces gradient variance and can yield stable learning.
- **Compute-efficiency and AutoRL scaling.** Because PPO typically performs multiple epochs per batch, AutoRL-style sweeps amplify PPO compute. REINFORCE offers a favorable trade-off when many configurations must be explored.

9 Discussion: implications for edge computing and sustainability

The strongest-performing configurations combine high tool-use ($\approx 95.7\%$) with near-zero violations and low normalized $|\lambda|$. Importantly, these results are achieved with an algorithmically simple policy-gradient method, reinforcing the argument that lightweight RL is a viable candidate for edge-focused PdM.

From a sustainability perspective, algorithmic simplicity matters: when AutoRL frameworks are used as practitioner-facing platforms, reducing the compute per experiment reduces the overall energy footprint of the exploration process [9]. This aligns with Industry 4.0/5.0 trajectories where operational excellence and environmental responsibility are jointly optimized.

10 Limitations and future work

This study is intentionally scoped to a dataset-driven episodic environment. Future work should:

- evaluate deployment-centric metrics (model size, inference latency, energy) on representative edge hardware,
- refine temporal-state handling in temporal extractors (explicit reset on episode boundaries),
- validate robustness under sensor noise, drift, and domain shift across machines.

11 Conclusion

We presented AutoRL, a reproducible pipeline for training and evaluating PdM RL agents with attention-based feature extraction. The experimental evidence on SIT milling data demonstrates that REINFORCE paired with temporal or multi-head attention is competitive with (and in aggregate outperforms) PPO and other baselines, while preserving algorithmic simplicity desirable for edge-computing and sustainable AI goals.

Code and data availability

This manuscript is based on the accompanying AutoRL codebase (`train_agent.py`, `rl_pdm.py`) and the evaluation artifacts under `results/Analysis/`.

References

[1] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[2] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[3] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[4] V. Mnih et al. Asynchronous methods for deep reinforcement learning. *Proceedings of ICML*, 2016.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

[6] A. Vaswani et al. Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] E. A. Nadaraya. On estimating regression. *Theory of Probability and Its Applications*, 9(1):141–142, 1964.

[8] G. S. Watson. Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A*, pages 359–372, 1964.

[9] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni. Green AI. *Communications of the ACM*, 63(12):54–63, 2020.

[10] E. Strubell, A. Ganesh, and A. McCallum. Energy and policy considerations for deep learning in NLP. *Proceedings of ACL*, 2019.

[11] R. Siraskar. An empirical study of the naïve REINFORCE algorithm for predictive maintenance. *Discover Applied Sciences*, 2025. DOI: 10.1007/s42452-025-06613-1.

[12] R. Siraskar, S. Kumar, S. Patil, A. Bongale, and K. Kotecha. Application of the Nadaraya–Watson estimator based attention mechanism to the field of predictive maintenance. *MethodsX*, 12:102754, 2024. DOI: 10.1016/j.mex.2024.102754.

[13] Reinforcement learning for predictive maintenance: a systematic technical review. *Artificial Intelligence Review*, 2023. DOI: 10.1007/s10462-023-10468-6.

[14] A. Raffin et al. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

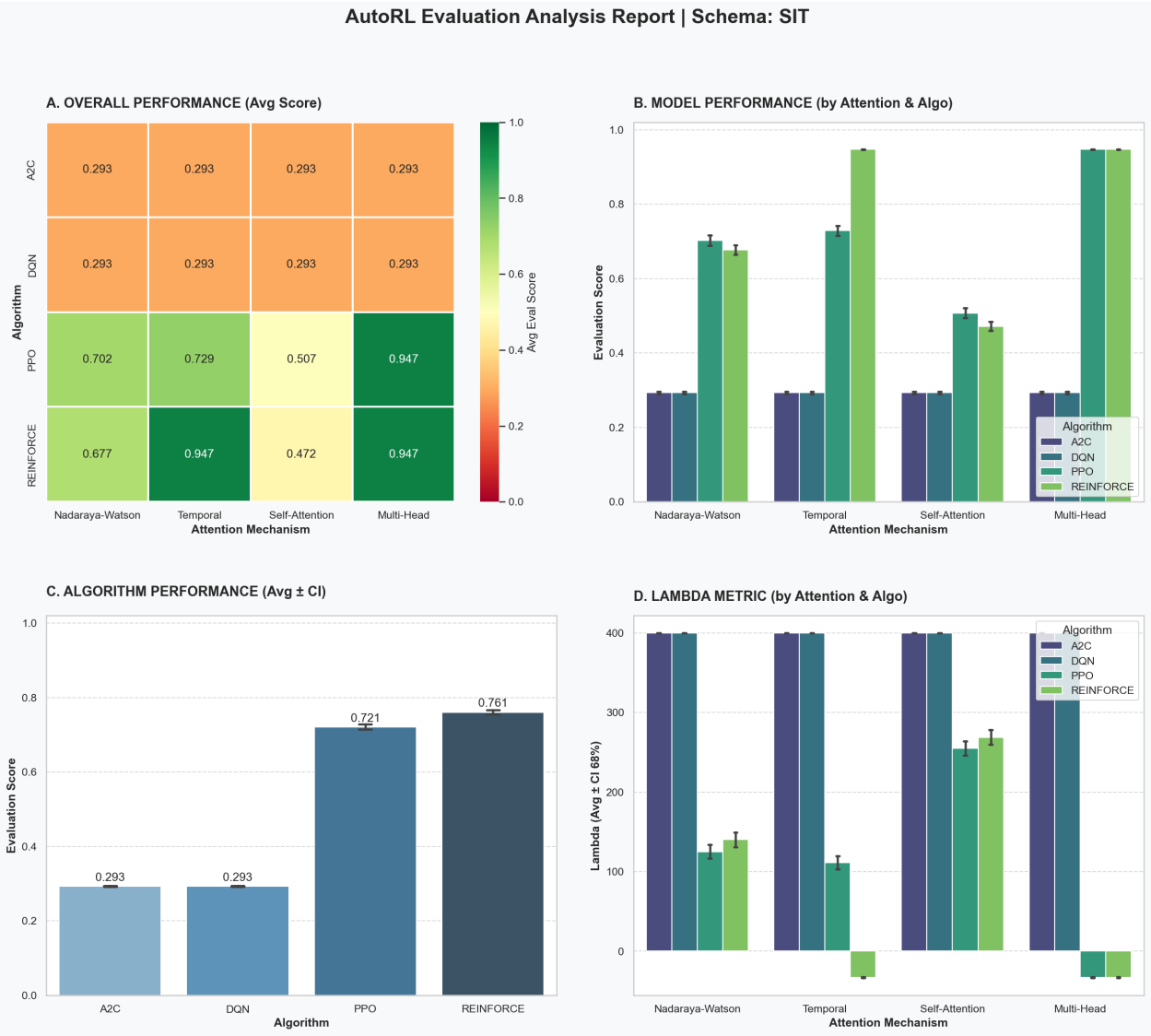
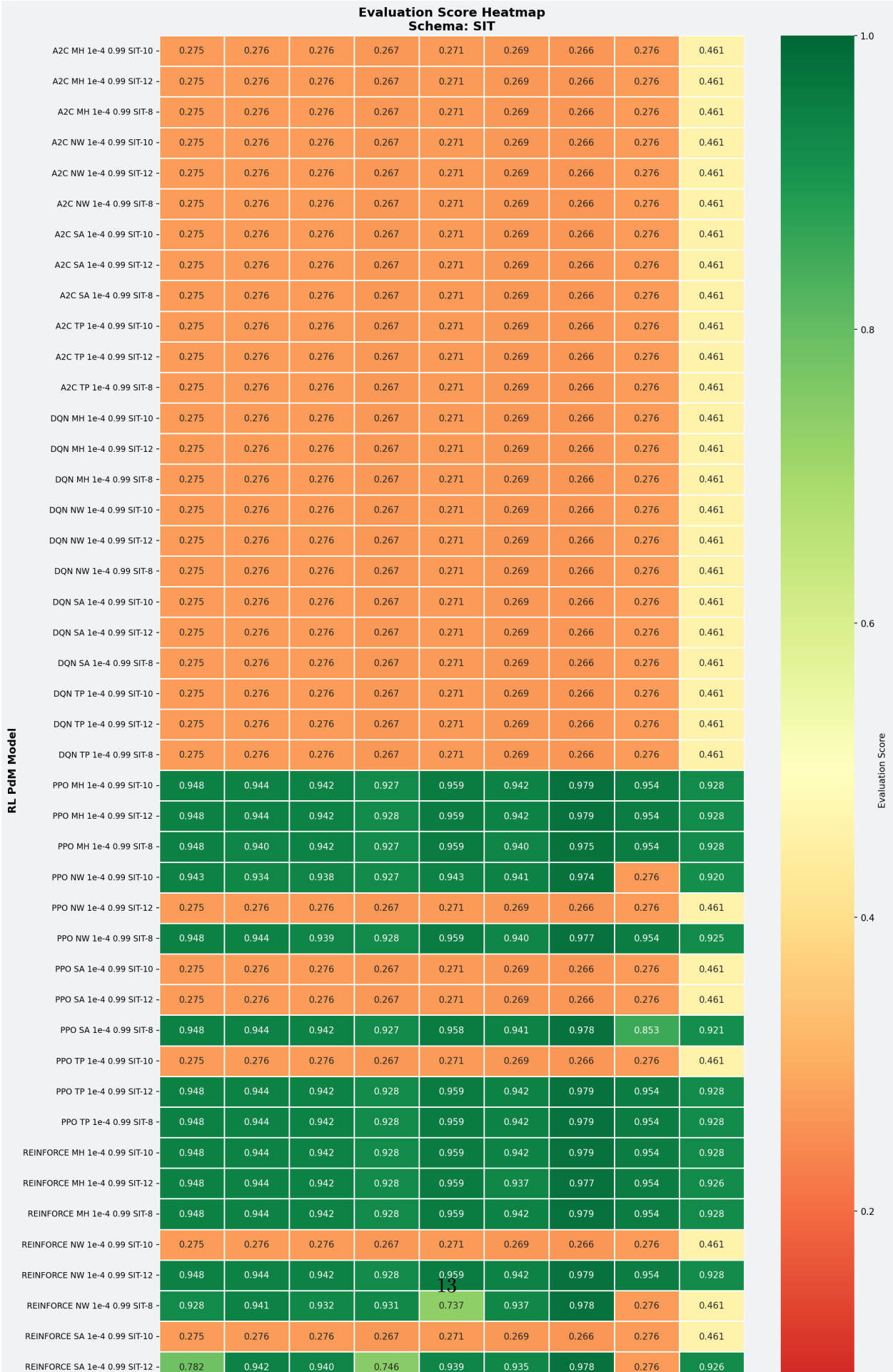


Figure 1: AutoRL evaluation analysis report (SIT schema). Panels summarize (A) mean evaluation score across algorithms and attention mechanisms, (B) grouped comparisons, (C) algorithm-level means with confidence intervals, and (D) the λ metric.


Evaluation Score Heatmap
Schema: SIT
RL PdM Model
Evaluation Score
1.0
0.8
0.6
0.4
0.2
A2C MH 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C MH 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C MH 1e-4 0.99 SIT-8 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C NW 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C NW 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C NW 1e-4 0.99 SIT-8 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C SA 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C SA 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C SA 1e-4 0.99 SIT-8 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C TP 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C TP 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
A2C TP 1e-4 0.99 SIT-8 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN MH 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN MH 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN MH 1e-4 0.99 SIT-8 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN NW 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN NW 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN NW 1e-4 0.99 SIT-8 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN SA 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN SA 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN SA 1e-4 0.99 SIT-8 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN TP 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN TP 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
DQN TP 1e-4 0.99 SIT-8 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
PPO MH 1e-4 0.99 SIT-10 0.948 0.944 0.942 0.927 0.959 0.942 0.979 0.954 0.928
PPO MH 1e-4 0.99 SIT-12 0.948 0.944 0.942 0.928 0.959 0.942 0.979 0.954 0.928
PPO MH 1e-4 0.99 SIT-8 0.948 0.940 0.942 0.927 0.959 0.940 0.975 0.954 0.928
PPO NW 1e-4 0.99 SIT-10 0.943 0.934 0.938 0.927 0.943 0.941 0.974 0.276 0.920
PPO NW 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
PPO NW 1e-4 0.99 SIT-8 0.948 0.944 0.939 0.928 0.959 0.940 0.977 0.954 0.925
PPO SA 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
PPO SA 1e-4 0.99 SIT-12 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
PPO SA 1e-4 0.99 SIT-8 0.948 0.944 0.942 0.927 0.958 0.941 0.978 0.853 0.921
PPO TP 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
PPO TP 1e-4 0.99 SIT-12 0.948 0.944 0.942 0.928 0.959 0.942 0.979 0.954 0.928
PPO TP 1e-4 0.99 SIT-8 0.948 0.944 0.942 0.928 0.959 0.942 0.979 0.954 0.928
REINFORCE MH 1e-4 0.99 SIT-10 0.948 0.944 0.942 0.928 0.959 0.942 0.979 0.954 0.928
REINFORCE MH 1e-4 0.99 SIT-12 0.948 0.944 0.942 0.928 0.959 0.937 0.977 0.954 0.926
REINFORCE MH 1e-4 0.99 SIT-8 0.948 0.944 0.942 0.928 0.959 0.942 0.979 0.954 0.928
REINFORCE NW 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
REINFORCE NW 1e-4 0.99 SIT-12 0.948 0.944 0.942 0.928 0.959 0.942 0.979 0.954 0.928
REINFORCE NW 1e-4 0.99 SIT-8 0.928 0.941 0.932 0.931 0.737 0.937 0.978 0.276 0.461
REINFORCE SA 1e-4 0.99 SIT-10 0.275 0.276 0.276 0.267 0.271 0.269 0.266 0.276 0.461
REINFORCE SA 1e-4 0.99 SIT-12 0.782 0.942 0.940 0.746 0.939 0.935 0.978 0.276 0.926